



BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

CSE 220 Project Proposal

1. Project Title

Electrocardiogram Analyzer: A Real-Time ECG Signal Analyzer

2. Team Information

Member 1: Sumaiya Sultana Labiba, 2305044

Member 2: Safa Tasnim, 2305056

3. Problem Description

Electrocardiogram (ECG) signals are one of the most widely used physiological measurements for monitoring heart health, but raw ECG recordings captured by real sensors and hospital equipment are rarely clean. They are typically corrupted by baseline wander (from patient breathing and body movement), 50/60 Hz powerline interference, and muscle artifact noise. This corruption makes both manual visual inspection and automated downstream analysis (such as heartbeat detection) unreliable if the raw signal is used directly.

This project addresses the problem of extracting reliable, clinically meaningful information -- clean waveform, heartbeat locations, and heart rate -- from noisy, real-world ECG recordings using classical signal processing techniques taught in this course, rather than opaque machine learning models. The intended users are students and instructors studying biomedical signal processing, and more broadly, this pipeline mirrors the front-end processing stage used in real cardiac monitoring devices and wearables before any diagnostic decision is made.

4. Proposed Solution

Input: Real, publicly available ECG recordings from the PhysioNet MIT-BIH Arrhythmia Database, which additionally provides expert cardiologist-annotated ground-truth R-peak locations for every recording. This allows us to not only detect heartbeats, but also quantitatively validate our detector's accuracy against real clinical annotations.

Processing Pipeline:

1. Preprocessing -- a bandpass/high-pass filter removes baseline wander, and a notch filter removes 50/60 Hz powerline interference.
2. QRS (heartbeat) Detection -- implemented using the classical Pan-Tompkins pipeline: bandpass filtering, derivative (slope) computation, squaring, moving-window integration, and adaptive thresholding to locate each heartbeat.
3. Frequency-Domain Analysis -- FFT is computed on both raw and filtered signals to visualize how the noise components are removed and to justify filter cutoff choices.

4. Heart Rate Estimation -- computed from the time interval between consecutive detected R-peaks (R-R intervals).

5. Validation -- detected R-peaks are compared against the database's ground-truth annotations to report precision and recall.

Output: A cleaned ECG waveform, an annotated waveform with detected heartbeats marked, a real-time heart rate readout, a frequency spectrum view, and a detection-accuracy report.

Block Diagram: Raw ECG -> Preprocessing Filters (baseline wander + powerline noise removal) -> QRS Detection (Pan-Tompkins) -> R-R Interval / Heart Rate Computation -> Visualization + Validation Dashboard, with an FFT branch running in parallel on both raw and filtered signals for spectral comparison.

5. Signal Processing Concepts Used

Convolution and LTI Systems -- all filtering stages (baseline wander removal, notch filtering, the Pan-Tompkins derivative and moving-window integration steps) are implemented as convolution with a designed impulse response, directly applying LTI system theory.

Filter Design (FIR/IIR) -- band-pass and notch filters are designed and their frequency response is visualized to justify cutoff selection, applying frequency response and pole-zero concepts.

Fourier Transform (FFT) -- used to inspect the frequency content of raw versus filtered signals, and to justify where noise energy is concentrated.

Sampling and Discrete-Time Signals -- ECG recordings are discrete-time sampled signals; R-R interval timing and heart rate estimation rely on correct handling of the sampling rate.

Thresholding / Decision Logic on Processed Signals -- adaptive thresholding on the integrated signal to localize heartbeats, a standard DSP technique for event detection.

(Stretch goal) Power Spectral Density -- used for heart rate variability (HRV) analysis in the frequency domain, if time permits.

6. User Interface Design

The interface will allow a user to select or upload an ECG recording, then view its analysis through several linked, synchronized panels:

1. Raw Signal Panel -- displays the unprocessed ECG waveform as recorded.

2. Filtered Signal Panel -- displays the cleaned waveform after baseline wander and powerline noise removal, updated live as the user adjusts filter cutoff-frequency sliders, so the effect of filtering is directly visible.

3. Heartbeat Detection Panel -- the filtered waveform with detected R-peaks marked, alongside a live-updating heart rate (BPM) readout.

4. Frequency Spectrum Panel -- side-by-side FFT magnitude plots of the raw versus filtered signal.

5. Validation Dashboard -- precision/recall of detected heartbeats against the database's expert-annotated ground truth, for the currently loaded recording.

The layout will use a tabbed or multi-panel dashboard so a user can move between recordings and immediately see how each processing stage transforms the signal.

7. Project Justification

This project is directly built on core CSE 220 topics: convolution and LTI system behavior (filter implementation), Fourier analysis (spectral inspection of noise and signal content), sampling theory (discrete-time signal handling), and filter design (cutoff selection and frequency response). Rather than treating these as isolated exercises, the project applies them together in a single, coherent real-world pipeline, from raw noisy sensor data to a validated clinical measurement.

The approximately six-week timeline is justified because the project has several distinct, non-trivial stages that must each be implemented, tested, and integrated with a live UI: filter design and validation, the multi-stage Pan-Tompkins detection algorithm, FFT-based spectral visualization, quantitative accuracy validation against ground-truth annotations, and a responsive multi-panel frontend with interactive controls. Additionally, because real ECG data will be used (rather than synthetic signals), meaningful time is needed for testing across multiple recordings with different noise characteristics and arrhythmia types.

8. Tools and Technologies

Backend: Python, with NumPy and SciPy for filter design, convolution, and FFT computation, and the wfdb package to directly read ECG records and annotations from the PhysioNet MIT-BIH Arrhythmia Database. FastAPI server will expose processed results to the frontend.

Frontend: React, with a charting library (e.g., Plotly.js or Recharts) for interactive, linked waveform and spectrum visualizations, and slider controls for live filter-parameter adjustment.

Data: PhysioNet MIT-BIH Arrhythmia Database (real, annotated ECG recordings, freely available for academic use).

Minimum Project Requirements

- Include one or more signal processing algorithm(s) as the core component.
- Include a user interface with meaningful visualization.
- Have sufficient scope for approximately six weeks of development.
- Be technically feasible within the available time and resources.